

Вопросы по курсу "Java".

1. Java core.

1.1 Что такое примитивные типы данных в Java?

- Перечислите все примитивные типы данных в Java, сколько байт они занимают?

Тип	Байт
byte	1
short	2
int	4
long	8
float	4
double	8
char	2
boolean	1 бит

- Чем примитивные типы отличаются от объектов?

Примитивы хранят значение, объекты — ссылку на данные в heap.

- В каких случаях лучше использовать примитив, а не обертку?

Частое создание оберток может вызывать лишние объекты и тормоза (особенно в циклах).

1.2 Что такое классы-обертки (Wrapper Classes)?

- Для чего нужны классы-обертки?

Позволяют использовать примитивы как объекты (например, в коллекциях: `List<Integer>`).

- Что такое autoboxing и unboxing?

```
1 Integer x = 5; // autoboxing
2 int y = x;     // unboxing
```

- В каких случаях использование классов-оберток может быть неэффективным?

Частое создание оберток может вызывать лишние объекты и тормоза (особенно в циклах).

1.3 Принципы ООП в Java.

4 основных принципа ООП.

- Инкапсуляция - это сокрытие внутренней реализации класса и предоставление доступа к данным через методы.
- Наследование - это возможность создавать новые классы на основе существующих, наследуя их свойства и методы.
- Полиморфизм - это возможность использовать объекты разных классов через единый интерфейс (например, переопределение методов).
- Абстракция - это процесс выделения общих характеристик объектов и сокрытие деталей реализации.

Подвопросы.

- Как инкапсуляция достигается через модификаторы доступа ('private', 'protected')?
Сокрывают доступа к методу.
- Приведите пример полиморфизма с интерфейсами или абстрактными классами.

```
1 interface Animal {  
2     void sound();  
3 }  
4 class Dog implements Animal {  
5     public void sound() { System.out.println("Woof"); }  
6 }  
7 Animal a = new Dog();  
8 a.sound(); // "Woof"
```

- Что такое наследование и как оно реализовано в Java?
Ключевое слово extends — подкласс наследует поля и методы суперкласса.

1.4 Интерфейсы vs. Абстрактные классы

- Какие возможности появились у интерфейсов в Java 8+ (default-методы, static-методы)?

Интерфейсы могут содержать реализацию методов с модификаторами default и static.

```
1 interface A {  
2     default void show() { System.out.println("Default"); }  
3     static void util() { System.out.println("Static"); }  
4 }
```

- Когда следует использовать интерфейс, а когда абстрактный класс?
Интерфейс — для описания поведения; абстрактный класс — если нужен базовый функционал и состояние (поля).

1.5 String vs StringBuilder vs StringBuffer

- Почему строки в Java являются неизменяемыми (immutable)?
Безопасность, потокобезопасность, возможность кеширования в пуле строк.
- Как работает пул строк (String Pool)?
Повторяющиеся строковые литералы хранятся один раз в специальной области памяти (heap, interned).

1.6 Устройство класса: конструктор, getter, setter, equals & hashCode, toString

- Почему нужно переопределять основные методы класса `Object`?
Для корректного сравнения объектов (`equals`), работы с коллекциями (`hashCode`), читаемого вывода (`toString`).

1.7 JVM vs JDK vs JRE

- JVM — исполняет Java-код.
- JRE — нужен, чтобы запускать программы.
- JDK — нужен, чтобы разрабатывать программы.

Подвопросы

- Какая из этих сред необходима для поставки на прод и почему?
Достаточно JRE или `runtime image`, так как она содержит только исполнение (JVM + библиотеки).

- Какие дополнительные инструменты Java есть в JDK?
Компилятор `javac`, отладчик `jdb`, документация `javadoc`, упаковщик `jar` и др.

1.8 Ключевое слово `static`

- Какие элементы в Java могут быть объявлены как `static`?
Поля, методы, блоки и вложенные классы.
- В чем разница между статическими и нестатическими методами?
`static` метод не требует создания объекта и не имеет доступа к нестатическим членам.
- Можно ли обращаться к нестатическим полям из статического метода?
Нет, требуется объект для доступа к нестатическим полям.

1.9 Ключевое слово `final`

- Какие элементы класса можно пометить как `final`?
Классы, методы и переменные.
- В чем разница между `final`, `finally` и `finalize`?
`final` — неизменяемость, `finally` — блок после `try-catch`, `finalize()` — устаревший метод очистки перед сборкой мусора.
- Почему `final` используется для создания иммутабельных классов?
Запрещает изменение полей и наследование, обеспечивая неизменность состояния.

1.10 Что такое перечисления (`enum`)

- Могут ли `enum` содержать поля и методы?
Да, можно добавлять поля, конструкторы и методы.

```

1 enum Color {
2     RED("FF0000");
3     private final String code;
4     Color(String code) { this.code = code; }
5     String getCode() { return code; }
6 }

```

- Почему для сравнения `enum` рекомендуется использовать `equals`, а не `==`?
Наоборот: `==` безопаснее, так как `enum` — синглтоны.

1.11 Что такое исключения (exceptions)

- Какие типы исключений существуют?
Checked (например, `IOException`) и Unchecked (`NullPointerException`, наследники `RuntimeException`).
- Как обрабатывать исключения с помощью `try-catch-finally`?

```

1 try {
2     int x = 1 / 0;
3 } catch (ArithmeticException e) {
4     System.out.println("Error");
5 } finally {
6     System.out.println("Done");
7 }

```

- Чем отличается `throw` от `throws`?
`throw` — выбрасывает исключение, `throws` — объявляет его в сигнатуре метода.

1.12 Что такое класс `Object`

- Какие основные методы есть в классе `Object`?
`equals()`, `hashCode()`, `toString()`, `clone()`, `finalize()`, `wait()`, `notify()`.

- Почему важно переопределять `equals` и `hashCode` вместе?
Чтобы объекты корректно работали в коллекциях с хэшированием (например, `HashMap`).
- Как работает метод `toString` и зачем его переопределять?
Возвращает строковое представление объекта; переопределение улучшает читаемость вывода.

1.13 Что такое Reflection API

Определение. Reflection API — это набор классов в пакете `java.lang.reflect`, который позволяет программе исследовать и изменять структуру классов, методов, полей и конструкторов во время выполнения (runtime).

- Как можно получить информацию о классе во время выполнения?
Через объект `Class`, например, `MyClass.class` или `obj.getClass()`.
- Какие есть ограничения у Reflection API?
Сниженная производительность, безопасность, отсутствие поддержки некоторых JVM оптимизаций.
- В каких случаях Reflection API может быть полезен?
Фреймворки, тестирование, динамическая загрузка классов и вызов методов.

1.14 Что такое Optional

- Как и зачем использовать `Optional` в Java?
Для явного указания отсутствия значения, избегая `null`.
- Какие методы есть в `Optional` и как их использовать?
`of()`, `empty()`, `isPresent()`, `orElse()`, `ifPresent()`.

- В каких случаях лучше возвращать `Optional`, а не `null`?

Когда результат может отсутствовать, и нужно избежать `NullPointerException`.

1.15 Какие есть альтернативы возврату `null` из методов

- Что такое Null Object Pattern?

Объект-заглушка, который ведет себя как "пустой" вместо `null`.

- Когда лучше кидать исключение, а не возвращать `Optional`?

При ошибках или некорректных ситуациях, когда отсутствует валидный результат.

- Какие еще подходы существуют для избежания `null`?

Использование `Optional`, Null Object Pattern, валидация входных данных.

1.16 Коллекции (Collections Framework) в Java

- Какие основные интерфейсы и классы есть в Collections Framework?

Интерфейсы: `Collection`, `List`, `Set`, `Queue`; классы: `ArrayList`, `LinkedList`, `HashSet`, `TreeSet`.

- Чем `ArrayList` отличается от `LinkedList`?

`ArrayList` — массивный список с быстрым доступом по индексу;
`LinkedList` — двусвязный список с быстрыми вставками/удалениями.

- Чем `List` отличается от `Set`?

`List` допускает дубликаты и упорядочен, `Set` — уникальные элементы без порядка (или с сортировкой).

1.17 ArrayList vs Обычный массив

- Как `ArrayList` увеличивает свой размер?
Создает новый массив с большим размером и копирует данные (обычно 1.5-2 раза больше).
- Почему `ArrayList` не потокобезопасен?
Нет внутренней синхронизации при изменении данных.
- Какие преимущества у `ArrayList` перед обычным массивом?
Динамический размер, удобные методы добавления, удаления и поиска.

1.18 Что такое Iterable и Iterator

- Как работает цикл `for-each` под капотом?
Использует `Iterator` для последовательного обхода коллекции.
- Можно ли удалять элементы из коллекции во время итерации?
Да, через метод `Iterator.remove()`, иначе возникнет `ConcurrentModificationException`.
- Чем `Iterable` отличается от `Iterator`?
`Iterable` предоставляет `Iterator`, а `Iterator` — сам объект для обхода.

1.19 Что такое анонимные классы и лямбда-выражения

- В каких сценариях каждый из них применять?
Анонимные классы — для единичных реализаций; лямбды — для компактного функционального кода.
- Как лямбда-выражения связаны с функциональными интерфейсами?
Лямбды реализуют функциональный интерфейс:

```

1 @FunctionalInterface
2 public interface Runnable {
3     public abstract void run();
4 }

```

- Какие недостатки у анонимных классов?

Более громоздкий синтаксис, создание отдельного класса в байткоде.

1.20 Лямбда-выражения и функциональные интерфейсы

- Как лямбда-выражения связаны с функциональными интерфейсами?

Лямбды реализуют функциональный интерфейс:

```

1 @FunctionalInterface
2 public interface Runnable {
3     public abstract void run();
4 }

```

- Какие есть стандартные функциональные интерфейсы (Predicate, Function, Consumer)?

— Predicate<T>

Принимает объект типа T, возвращает `boolean`. Часто используется для фильтрации.

```

1 Predicate<String> isEmpty = s -> s.isEmpty();
2 System.out.println(isEmpty.test("")); // true

```

— Function<T, R>

Принимает T, возвращает R. Применяется для преобразования значений.

```

1 Function<String, Integer> toLength = s -> s.length();
2 System.out.println(toLength.apply("abc")); // 3

```

– Consumer<T>

Принимает T, ничего не возвращает. Используется для выполнения действий с объектом.

```
1 Consumer<String> print = s -> System.out.println(s);  
2 print.accept("Hello"); // Hello
```

– Supplier<T>

Ничего не принимает, возвращает T. Удобен для ленивой инициализации.

```
1 Supplier<String> getName = () -> "Java";  
2 System.out.println(getName.get()); // Java
```

– UnaryOperator<T>

Специализация Function<T, T>, принимает и возвращает один тип.

```
1 UnaryOperator<Integer> square = x -> x * x;  
2 System.out.println(square.apply(5)); // 25
```

– BinaryOperator<T>

Специализация BiFunction<T, T, T>, принимает два аргумента одного типа, возвращает тот же тип.

```
1 BinaryOperator<Integer> add = (a, b) -> a + b;  
2 System.out.println(add.apply(2, 3)); // 5
```

- В чем преимущества лямбда-выражений?

Краткость, читаемость, поддержка функционального программирования.

2. Java Advanced.

2.1 Что такое generics (обобщения)

- Для чего нужны generics?

Позволяют создавать обобщённый код, безопасный по типам, без кастов: List<String>, Map<K, V>.

- В чем разница между `List<String>` и `List<Object>`?
`List<String>` принимает только строки; `List<Object>` — объекты любых типов.
- Что такое wildcards (`?`, `? extends T`, `? super T`)?
`?` — неизвестный тип; `? extends T` — любой подтип `T`; `? super T` — любой супертип `T`.

2.2 Почему нельзя создать массив дженериков (`new T[]`)?

- Какие есть обходные пути?
Использовать `List<T>`, `Array.newInstance(Class<T>, size)` или приведение с подавлением предупреждения.
- Как связаны массивы и дженерики в Java?
Массивы — ковариантны и проверяются во время выполнения, дженерики — инвариантны и проверяются на этапе компиляции.

2.3 Как работает HashMap

- Как разрешаются коллизии в HashMap?
Через цепочки (связанный список или дерево с Java 8+).
- Почему важно правильно переопределять `equals()` и `hashCode()` для ключей?
От `hashCode()` зависит индекс корзины, от `equals()` — корректность поиска.

2.4 HashSet vs LinkedHashSet

- Как HashSet проверяет уникальность элементов?
Через `hashCode()` и `equals()`.

- Почему порядок элементов в `HashSet` не гарантируется?
Порядок зависит от хеш-функции и внутренней структуры.
- Чем `LinkedHashSet` отличается от `HashSet`?
`LinkedHashSet` сохраняет порядок добавления элементов.

2.5 Как происходит работа с многопоточностью в Java

- Как создать поток (`Thread`) в Java?
Через наследование от `Thread` или реализацию интерфейса `Runnable`.

```
1 new Thread(() -> System.out.println("Hi")).start();
```

- Что такое `synchronized` и как он работает?
Блокирует доступ к ресурсу — только один поток может войти в синхронизированный блок одновременно.
- Какие есть способы синхронизации потоков?
Ключевое слово `synchronized/volatile`, классы из пакета `java.util.concurrent` (`Lock`, `Semaphore`, `CountDownLatch`).

2.6 Что такое Stream API

- Какие есть основные операции в Stream API (`map`, `filter`, `reduce`)?
`map()` — преобразование, `filter()` — отбор, `reduce()` — свертка к одному значению.
- В чем разница между промежуточными и терминальными операциями?
Промежуточные возвращают `Stream`, терминальные завершают вычисление (например, `collect()`, `forEach()`).
- Как Stream API работает с параллельными потоками?
`parallelStream()` делит данные на подзадачи, исполняет их параллельно с помощью `ForkJoinPool`.

2.7 Что такое аннотации в Java, как создать собственную

```
1 @Target(ElementType.METHOD)
2 @Retention(RetentionPolicy.RUNTIME)
3 public @interface MyAnnotation {
4     String value();
5     int count() default 1;
6 }
```

- Какие есть стандартные аннотации (@Override, @Deprecated)?
@Override — проверка переопределения, @Deprecated — пометка устаревших методов.
- Как обрабатывать аннотации с помощью Reflection API?
Через методы `Class.getAnnotations()`, `Method.getAnnotation(Class)` и др.

2.8 Что такое Records

- Чем Records отличаются от обычных классов?
Автоматически генерируют поля, конструктор, `equals()`, `hashCode()`, `toString()`.
- Какие методы автоматически генерируются для Records?
Все стандартные: `constructor`, `getters`, `equals()`, `hashCode()`, `toString()`.
- В каких случаях удобно использовать Records?
Для DTO и неизменяемых структур данных.

2.9 Устройство памяти в JVM: области и их ответственность

- Как устроена память в JVM (Heap, Stack, Method Area)?

Heap — область для хранения объектов.

Stack — для хранения вызовов методов и локальных переменных.

Method Area — для хранения информации о классах (структура, константы, статические поля).

- Чем отличается Heap от Stack?

Heap управляется сборщиком мусора, общий для всех потоков.

Stack — индивидуален для каждого потока, быстрее по доступу, управляется вручную (при вызове/выходе из метода).

- Какие объекты хранятся в Heap, а какие в Stack?

В **Heap** — все объекты и их поля. В **Stack** — ссылки на объекты, примитивные переменные, контекст выполнения методов.

2.10 Устройство Garbage Collection в JVM: задачи, виды, как работает

- Как работает Garbage Collection в Java?

Автоматически освобождает память, удаляя объекты, на которые нет активных ссылок.

- Какие бывают этапы сборки мусора (Minor GC, Major GC, Full GC)?

Minor GC — чистка молодого поколения (Young Gen), быстрая.

Major GC — чистка старого поколения (Old Gen), медленнее.

Full GC — сборка всего кучи, включая метainформацию, самая дорогая по времени.

- Что такое Generational GC и как он работает?

Модель поколений: объекты сначала попадают в **Young Gen**, затем — в

Old Gen, если живут долго. Это повышает эффективность сборки мусора.

2.11 Что такое JIT-компиляция

JIT-компиляция (Just-In-Time compilation) — это технология в JVM, при которой байт-код Java компилируется в машинный код не заранее, а прямо во время выполнения программы (в нужный момент).

- Как JIT-компилятор ускоряет выполнение Java-кода?
JIT (Just-In-Time) компилирует байткод в машинный код во время выполнения, ускоряя повторяющиеся участки.
- Чем интерпретатор отличается от JIT-компилятора?
Интерпретатор выполняет байткод построчно. JIT компилирует часто используемые методы в машинный код один раз и повторно использует его.
- Какова единица компиляции в Java?
Обычно — **метод**. JIT компилирует метод целиком.

2.12 Что такое Unit и интеграционные тесты

- Какие задачи решают Unit-тесты?
Проверяют корректность работы отдельных методов или классов в изоляции.
- Почему важно разделять Unit-тесты и интеграционные тесты?
Unit-тесты быстрые и изолированные. Интеграционные проверяют взаимодействие компонентов и требуют окружения (БД, сеть и т.д.).
- Какой фреймворк используется для написания Unit-тестов в Java?
JUnit — самый популярный. Также: TestNG, Mockito (для моков).

2.13 Что такое E2E тестирование

- Чем оно отличается от модульного и интеграционного?
E2E (end-to-end) тестирует всю систему целиком — от пользовательского интерфейса до БД.
- Какие слои приложения охватывает E2E тестирование?
UI, backend, база данных, внешние сервисы — вся цепочка обработки запроса.
- Почему E2E тесты считаются наиболее приближенными к реальным условиям?
Они эмулируют поведение настоящего пользователя, тестируя всю систему в реальной среде.

2.14 Что такое потокобезопасность и как ее достичь

- Какие классы являются потокобезопасными по умолчанию?
Vector, Hashtable, ConcurrentHashMap, AtomicInteger и другие из пакета `java.util.concurrent`.
- Почему классы без состояния (stateless) всегда потокобезопасны?
Они не хранят данные между вызовами, не используют общую изменяемую память — значит, не возникает конфликтов между потоками.

2.15 Проблема Race Condition

- Как состояние гонки может привести к некорректной работе программы?
Если два потока одновременно обращаются к общей переменной, результат зависит от порядка выполнения — это может вызвать ошибки, дубли, потерю данных.

- Приведите пример, где возникает race condition, и объясните, как его можно исправить.

```
1 int counter = 0;
2 Runnable r = () -> {
3     for (int i = 0; i < 1000; i++) {
4         counter++; // race condition
5     }
6 };
```

Исправление — синхронизация:

```
1 synchronized (lock) {
2     counter++;
3 }
```

Или использовать `AtomicInteger`.

2.16 Ключевое слово `volatile`

- Когда следует использовать `volatile` и какие проблемы он решает?
`volatile` обеспечивает видимость изменений переменной между потоками. Используется, когда один поток записывает значение, а другие читают.
- Почему `volatile` не решает проблему атомарности?
`volatile` не предотвращает гонки при чтении-модификации-записи (например, `x++`), т.к. такие операции не атомарны.
- В каких случаях использование `volatile` достаточно, а в каких — нет?
Достаточно: флаг завершения потока (`volatile boolean running`).
Недостаточно: счётчики, инкременты, кэш с множественными полями — нужен `synchronized` или атомарные классы.

2.17 Ключевое слово `synchronized`

- Как работает ключевое слово `synchronized`?

Оно устанавливает взаимное исключение (`mutex`), позволяя только одному потоку выполнять защищённый блок кода. Обеспечивает атомарность и видимость изменений.

- Какие объекты можно использовать в качестве монитора для `synchronized`?

Любой объект: `this`, класс (`MyClass.class`), любой объект-синхронизатор.

- В чем разница между синхронизированным методом и синхронизированным блоком?

Метод — синхронизация на `this` или `Class.class`. Блок — можно явно указать объект-монитор и защитить только часть кода.

2.18 Что такое атомарные классы

- Как работают атомарные классы (например, `AtomicLong`)?

Используют неблокирующие операции на основе `CAS` (`Compare-And-Swap`), предоставляя атомарность без блокировок.

- В каких сценариях атомарные классы предпочтительнее `synchronized`?

При частых коротких операциях (инкремент, счетчик), где накладные расходы на `synchronized` избыточны.

2.19 Что такое `Happens-before` в Java Memory Model (JMM)

`Happens-before` — это ключевое понятие в Java Memory Model (JMM), которое описывает гарантии порядка видимости операций между потоками. Если опе-

рация А happens-before операция В, то В увидит результат А или состояние, которое А оставила.

- Как happens-before гарантирует видимость изменений между потоками?
Если между двумя действиями есть отношение happens-before, все изменения, сделанные первым действием, будут видны второму.
- Примеры happens-before отношений:
 - Запись в `volatile` переменную → чтение из неё в другом потоке.
 - Вход в `synchronized` блок → выход из блока.
 - Запуск потока через `start()` → начало его `run()` метода.
 - Завершение `Thread.join()` → завершение потока.

2.20 Как создавать потоки в Java

С помощью класса `Thread`.

- Что такое `synchronized` и как он работает?
Ключевое слово для обеспечения взаимного исключения. Гарантирует, что только один поток выполнит блок кода одновременно.
- Какие есть способы синхронизации потоков?
 - `synchronized` (блоки и методы)
 - Классы из `java.util.concurrent`: `Lock`, `ReentrantLock`, `Semaphore`, `CountDownLatch`, `CyclicBarrier`
 - Атомарные переменные: `AtomicInteger`, `AtomicReference`
 - `volatile` — для простой видимости

2.21 Как Garbage Collector работает с Heap

- Чем отличается Heap от Stack?

Heap — это область памяти, где хранятся все объекты, созданные в процессе выполнения программы. Память в Heap общая для всех потоков и управляется сборщиком мусора.

Stack — память, выделенная для каждого потока отдельно, где хранятся локальные переменные, параметры методов и структура вызовов. Stack управляется автоматически при вызове и выходе из методов.

- Какие объекты хранятся в Heap, а какие в Stack?

В Heap хранятся все объекты и массивы, а также данные классов. В Stack хранятся примитивные локальные переменные и ссылки на объекты из Heap, используемые текущим потоком.

2.22 Как работает ConcurrentHashMap

- Какие операции в ConcurrentHashMap являются атомарными?

Атомарными являются операции `putIfAbsent()`, `remove(key, value)`, `replace(key, oldValue, newValue)`. Они обеспечивают потокобезопасное добавление, удаление и замену без внешней синхронизации.

- Какие еще есть потокобезопасные коллекции?

`CopyOnWriteArrayList`, `ConcurrentLinkedQueue`, `BlockingQueue` (`ArrayBlockingQueue`, `LinkedBlockingQueue`), `ConcurrentSkipListMap` и др.

2.23 Как работает CopyOnWriteArrayList

- Принцип работы CopyOnWriteArrayList:

При изменении (добавлении/удалении) создаётся новая копия внутреннего массива. Это позволяет избежать блокировок при чтении, обеспечивая

очень быструю потокобезопасную работу для сценариев с преобладанием чтений.

- Какие еще есть потокобезопасные коллекции?

См. выше: `ConcurrentHashMap`, `ConcurrentLinkedQueue`, `BlockingQueue`, `ConcurrentSkipListSet` и др.

2.24 Что такое `ExecutorService`, какие существуют реализации

- Разница между `newFixedThreadPool`, `newCachedThreadPool` и `newScheduledThreadPool`:

`newFixedThreadPool` — фиксированное количество потоков, переиспользуемых для задач.

`newCachedThreadPool` — динамическое создание и удаление потоков, подходит для большого числа коротких задач.

`newScheduledThreadPool` — пул для планирования задач с задержкой или периодическим выполнением.

- Как выбрать подходящий пул потоков?

Фиксированный пул — для постоянной нагрузки.

Cached — для непредсказуемых, кратковременных задач.

Scheduled — для задач с таймерами и расписанием.

2.25 `Future` vs `CompletableFuture`

- Методы и ограничения `Future`:

Методы: `get()`, `cancel()`, `isDone()`, `isCancelled()`.

Ограничения: `get()` блокирует поток, отсутствует поддержка цепочек и колбеков.

- Как `CompletableFuture` решает проблемы `Future`?

Добавляет асинхронные колбеки (`thenApply`, `thenAccept`), композицию задач, обработку ошибок, неблокирующее ожидание и поддержку цепочек задач.

3. DB.

3.1 Реляционные базы данных (SQL)

- Что такое реляционные базы данных? Реляционная база данных — это структура, в которой данные хранятся в виде связанных таблиц. Каждая таблица содержит строки (записи) и столбцы (поля).
- Основные понятия: **Таблица (table)** — двумерная структура с данными. **Связи (relations)** — логические связи между таблицами: `one-to-one`, `one-to-many`, `many-to-many`. **Индекс (index)** — структура для ускорения поиска. **Ключи**: `PRIMARY KEY` — уникальный идентификатор записи; `FOREIGN KEY` — ссылка на внешний ключ другой таблицы.
- Разница между SQL и NoSQL: SQL — структурированные данные, фиксированная схема, поддержка сложных запросов (`JOIN` и др.). NoSQL — гибкая схема, масштабируемость, разные модели хранения (документы, ключ-значение и т.д.).
- Популярные СУБД: **PostgreSQL** — мощная, с поддержкой расширений, транзакций, JSON. **MySQL** — широко используется, быстрая, простая. Также: SQLite, Oracle, MS SQL Server.

3.2 NoSQL базы данных

- Что такое NoSQL? NoSQL — это семейство СУБД, не использующих традиционную реляционную модель. Характеризуются горизонтальной масштабируемостью и гибкой схемой.

- Типы NoSQL:
 - **Документные** — MongoDB, CouchDB. Данные хранятся в виде документов (обычно JSON/BSON).
 - **Ключ-значение** — Redis, DynamoDB. Простейшая модель: ключ → значение.
 - **Графовые** — Neo4j. Хранят вершины и связи между ними.
 - **Колонно-ориентированные** — Cassandra, HBase. Эффективны при работе с большими объемами данных.
- Отличие от реляционных СУБД: NoSQL не требует жёсткой схемы, проще масштабируется, но хуже поддерживает сложные связи и транзакции.
- Когда NoSQL лучше SQL:
 - Большие объемы неструктурированных или слабо структурированных данных.
 - Высокая нагрузка на чтение/запись.
 - Нет необходимости в сложных связях и транзакциях.

3.3 Основные понятия в SQL, виды запросов

- Основные SQL-запросы: **SELECT** — выборка данных. **INSERT** — добавление записей. **UPDATE** — изменение данных. **DELETE** — удаление записей.
- Как работает JOIN? JOIN объединяет строки из двух таблиц по связанным полям:
 - **INNER JOIN** — только совпадающие записи.
 - **LEFT JOIN** — все записи из левой таблицы и совпадения из правой.
 - **RIGHT JOIN, FULL OUTER JOIN** — аналогично.
- Что такое подзапросы (subqueries)? Это запрос внутри другого запроса.

3.4 Транзакции и их свойства

- Что такое транзакция? Последовательность операций с данными, которая выполняется как единое целое.
- Свойства транзакций (ACID):
 - Atomicity (атомарность) – выполнены либо все подоперации, либо никакие.
 - Consistency (согласованность) – каждая успешная транзакция фиксирует только допустимые результаты.
 - Isolation (изолированность) – параллельные транзакции не влияют на результаты друг друга.
 - Durability (долговечность) – вне зависимости от сбоев системы результаты успешных транзакций сохраняются в системе
- Уровни изоляции транзакций:

READ UNCOMMITTED	Чтение незафиксированных данных.
READ COMMITTED	Чтение фиксированных данных.
REPEATABLE READ	Повторяемость чтения.
SERIALIZABLE	Упорядочиваемость.

- Потерянное обновление - изменение одного блока данных несколькими транзакциями.
 - “Грязное” чтение - чтение данных, измененных впоследствии откатившейся транзакцией.
 - неповторяющееся чтение - повторное чтение измененных данных одной и той же транзакцией.
 - Чтение фантомов - взаимосвязанные критерии изменения данных двумя транзакциями.
- Как завершить транзакцию: COMMIT — подтвердить изменения. ROLLBACK — откатить транзакцию.

3.5 Что такое ORM (Object-Relational Mapping)

- Как ORM преобразует объекты в таблицы? ORM отображает классы и их поля на таблицы и колонки в реляционной БД. Объекты сохраняются в БД автоматически, например, класс `User` становится таблицей `users`.
- Какие есть популярные ORM-фреймворки? Hibernate, JPA (спецификация), Spring Data JPA, MyBatis.

3.6 Что такое миграции базы данных

- Какие есть инструменты для миграций? Flyway, Liquibase — позволяют версионировать схему БД и управлять изменениями.

3.7 Что такое индексы, какие бывают

- Как индексы ускоряют запросы? Позволяют быстро находить строки по ключу (аналогично оглавлению), избегая полного сканирования таблицы.
- Какие недостатки есть у индексов? Замедление вставки/обновления данных, увеличение размера БД.

3.8 Что такое JDBC

JDBC — это низкоуровневый способ работы с базой данных. Он предоставляет «сырой» доступ к SQL и требует писать много кода вручную. ORM-фреймворки, такие как Hibernate, работают поверх JDBC, абстрагируя от него.

- Как выполнить SQL-запрос через JDBC? Через `Connection` создаётся `Statement` или `PreparedStatement`, выполняется `executeQuery()`, результаты читаются через `ResultSet`.

- В чем разница между `Statement` и `PreparedStatement`? `PreparedStatement` компилируется заранее, безопасен от SQL-инъекций, эффективен при многократных вызовах.

3.9 Что такое Primary Key

- Какие ограничения накладываются на колонку с Primary Key? Значения должны быть уникальными и не NULL.
- Можно ли изменить значение Primary Key после его создания? Технически — да, но это может нарушить связи, если ключ используется как внешний в других таблицах.

3.10 Что такое Foreign Key

- Что произойдет, если попытаться удалить запись, на которую ссылается Foreign Key? Будет ошибка, если не задано поведение (`ON DELETE CASCADE`, `SET NULL` и др.).
- Как Foreign Key помогает избежать "висячих" ссылок? Он гарантирует, что значения в столбце-ссылке существуют в родительской таблице.

3.11 Создание таблицы в SQL

- Какие типы данных можно использовать для колонок? `INT`, `VARCHAR`, `TEXT`, `BOOLEAN`, `DATE`, `TIMESTAMP` и др.
- Как добавить ограничение `NOT NULL` или `PRIMARY KEY`? Пример:
`CREATE TABLE users (id INT PRIMARY KEY, name VARCHAR(50) NOT NULL);`

3.12 Типы JOIN

- В чем разница между INNER JOIN и LEFT JOIN? INNER JOIN — только совпадающие строки. LEFT JOIN — все из левой таблицы, даже если нет совпадений.

3.13 Как работает GROUP BY в SQL

- Для чего используется COUNT() с GROUP BY? Для подсчета количества строк в каждой группе.
- Пример запроса: `SELECT country, COUNT(*) FROM users GROUP BY country;`

3.14 ACID

- Пример нарушения Atomicity: Перевод денег с одного счёта на другой: если деньги списались, но не зачислились — нарушена атомарность, пользователь теряет деньги.

3.15 Optimistic и Pessimistic блокировки

Пессимистическая блокировка - считаем, что конфликт вероятен, поэтому блокируем запись заранее.

Оптимистическая блокировка - считаем, что конфликт маловероятен, проверяем при сохранении.

3.16 CAP-теорема

- Consistency. При чтении гарантированно будут получены наиболее актуальные данные, либо ошибка.

- Availability. Любой запрос к распределённой системе завершается корректным откликом, но без гарантии актуальности данных.
- Partitioning. Система работает несмотря на потери / задержки в передачи сообщений между узлами.
- Какие два свойства выбирают чаще всего? В распределённых БД — AP (доступность + устойчивость к разделению) или CP (согласованность + устойчивость).

3.17 BASE в NoSQL

ACID → BASE.

- Basic Available (базовая доступность).
 - Система обеспечивает базовую доступность, т. е. каждый запрос будет обязательно завершён, успешно или нет;
 - Сбой части системы приводит к отказу в обслуживании только части пользователей.
- Soft State (неустойчивое состояние).
 - Система пребывает в гибком состоянии — очередность записей соблюдать необязательно, реплики могут какое-то время находиться в несогласованном состоянии, а система может самостоятельно изменяться для достижения согласованности;
 - Отказ от гарантий хранения промежуточных данных.
- Eventually Consistent (итоговая согласованность).
 - Все данные всё равно достигнут согласованности.
 - Существуют моменты в работе системы, когда можно видеть несогласованные данные.

Подвопросы.

- Как BASE отличается от ACID? BASE допускает eventual consistency, отказ от полной согласованности ради доступности и масштабируемости.
- Когда BASE предпочтительнее? Когда допустимы временные несогласованности — социальные сети, логи, кэши.

3.18 Репликация в БД

- Проблемы: Запаздывания (lag), конфликты при записи в несколько реплик, потеря согласованности.
- Разница между репликацией и шардированием: Репликация — копии одних и тех же данных. Шардирование — разделение данных между узлами.

3.19 Apache Cassandra

- Как обеспечивает отказоустойчивость и масштабируемость? Данные дублируются на несколько узлов, нет единой точки отказа, распределённое хранилище.
- Для каких типов данных подходит? Логи, сенсорные данные, временные ряды, данные с высокой скоростью записи и большого объема.

3.20 Cassandra vs. ScyllaDB

- В чем разница? ScyllaDB переписана на C++ (Cassandra — Java), использует асинхронную модель на основе **Seastar**.
- Почему ScyllaDB быстрее? Меньше задержек, лучше использование ресурсов.

- Как влияет архитектура «shard per core»? Каждое ядро CPU обслуживает свой shard (часть данных) — нет блокировок, масштабируемость по ядрам.

3.21 Keyspace и Column Family в Cassandra

- Что такое Primary Key? Уникальный идентификатор строки в таблице. Состоит из partition key и (опционально) clustering key.
- Partition key — что это? Определяет, на каком узле будет храниться строка. Строки с одинаковым ключом — на одном узле.
- Clustering key — что это? Определяет порядок строк внутри одного partition (аналог сортировки по колонке).

4. Spring, Мониторинг.

4.1 AOP в Spring

- Какие основные понятия AOP (аспект, совет, точка среза, соединение)?
Аспект (Aspect) — модуль кода, содержащий кросс-секциональную функциональность (например, логирование).
Совет (Advice) — действие, выполняемое в определённой точке выполнения (например, перед методом).
Точка среза (Pointcut) — выражение, указывающее, где должен применяться совет.
Соединение (Join point) — конкретная точка во времени исполнения (например, вызов метода).
- Какие типы советов существуют в Spring AOP?

- `@Before` — вызывается до выполнения метода.
- `@After` — вызывается после выполнения (в любом случае).
- `@AfterReturning` — вызывается при успешном завершении метода.
- `@AfterThrowing` — вызывается при исключении.
- `@Around` — оборачивает выполнение метода, даёт полный контроль.

4.2 IoC в Spring

Инверсия управления — это принцип, при котором управление зависимостями и созданием объектов передаётся фреймворку, а не осуществляется вручную в коде.

- Чем отличается управление объектами через `new` от IoC-контейнера?
Создание через `new` означает ручное управление зависимостями. IoC-контейнер создаёт и управляет объектами, внедряет зависимости автоматически.
- Какие способы внедрения зависимостей поддерживает Spring?
 - Через конструктор (рекомендуется).
 - Через сеттеры.
 - Через поля (`@Autowired`, нежелательно из-за невозможности тестирования).

4.3 Что такое Spring Bean

- Какие аннотации используются для создания бинов?
`@Component` — базовая аннотация.
`@Service`, `@Repository`, `@Controller` — специализации для конкретных слоёв.

- В чём разница между `@Autowired` и внедрением через конструктор?
`@Autowired` можно использовать для полей, сеттеров или конструкторов. Внедрение через конструктор предпочтительнее: облегчает тестирование и делает зависимости явными.

4.4 Жизненный цикл Spring Bean

- Как работают аннотации `@PostConstruct` и `@PreDestroy`?
`@PostConstruct` — метод вызывается сразу после создания и внедрения зависимостей.
`@PreDestroy` — метод вызывается перед уничтожением бина (только для `singleton`).
- Почему для бинов с `@Scope("prototype")` не вызывается `@PreDestroy`?
Контейнер не отслеживает `prototype`-бины после их создания, поэтому не вызывает метод `@PreDestroy`.

4.5 Scope бинов

В Spring `scope` (область видимости) бина определяет, как часто и в каком контексте создаются объекты, управляемые Spring-контейнером. По умолчанию все бины — `singleton`, но есть и другие области видимости, особенно актуальные для веб-приложений.

- Чем отличается `singleton` от `prototype`?
`singleton` — один бин на контейнер.
`prototype` — каждый запрос даёт новый экземпляр.
- Когда следует использовать `request` и `session` scope?
`request` — бин создаётся на каждый HTTP-запрос (например, для DTO).
`session` — бин живёт на протяжении всей сессии пользователя.

4.6 Что такое DispatcherServlet

- Какие компоненты участвуют в обработке HTTP-запроса?

DispatcherServlet — центральная точка, обрабатывающая HTTP-запросы. Он взаимодействует со следующими компонентами:

- **HandlerMapping** — определяет, какой контроллер должен обработать запрос.
 - **HandlerAdapter** — вызывает соответствующий метод контроллера.
 - **ViewResolver** — определяет, какое представление (View) отобразить.
 - **View** — рендерит HTML-ответ (например, JSP, Thymeleaf).
- Как работает связь между контроллером и представлением (View)?
Контроллер возвращает имя представления или объект **ModelAndView**. **ViewResolver** находит нужный шаблон, а **Model** передаёт в него данные.

4.7 Мappings HTTP-запросов в Spring

- Какие аннотации используются для маппинга HTTP-запросов?

– **@GetMapping**, **@PostMapping**, **@PutMapping**, **@DeleteMapping** — сокращения для **@RequestMapping(method = ...)**.

- Как извлечь параметр из URL с помощью **@PathVariable**?

Пример:

```
1 @GetMapping("/user/{id}")
2 public String getUser(@PathVariable Long id) {
3     ...
4 }
```

- Чем отличается **@RequestParam** от **@ModelAttribute**?

@RequestParam извлекает отдельный параметр запроса (например,

?name=...).

@ModelAttribute используется для связывания всех параметров с объектом (например, из формы).

4.8 Валидация в Spring MVC

- Как добавить валидацию с использованием Hibernate Validator?

Добавить аннотации к полям модели: @NotBlank, @Email, @Size, и т.д.

Пример:

```
1 public class User {
2     @NotBlank
3     private String name;
4
5     @Email
6     private String email;
7 }
```

- Как обработать ошибки валидации в контроллере?

Добавить аргумент BindingResult после объекта:

```
1 @PostMapping("/submit")
2 public String submit(@Valid User user, BindingResult result) {
3     if (result.hasErrors()) {
4         return "form";
5     }
6     return "success";
7 }
```

4.9 Обработка исключений в контроллерах Spring

- Как обработать исключение локально с помощью @ExceptionHandler?

Добавить метод в контроллер:

```
1 @ExceptionHandler(ResourceNotFoundException.class)
2 public ResponseEntity<String> handleNotFound() {
3     return ResponseEntity.status(404).body("Not Found");
4 }
```

- Что такое `@ControllerAdvice` и для чего он используется?
`@ControllerAdvice` позволяет централизованно обрабатывать исключения, применять `@ExceptionHandler` ко всем контроллерам.
- Как вернуть клиенту JSON-ответ при ошибке?
Использовать аннотацию `@ResponseBody` или вернуть `ResponseBody<...>` из метода-обработчика.

4.10 Что такое интерцепторы в Spring

В Spring интерцепторы (interceptors) — это компоненты, которые позволяют перехватывать HTTP-запросы до и после обработки их контроллерами. Они похожи на фильтры, но работают на уровне Spring MVC и дают более тонкий контроль над обработкой запросов.

- Для каких задач используются интерцепторы в Spring MVC?
Для перехвата и обработки запросов до и после вызова контроллера: логирование, аутентификация, измерение времени выполнения и т.д.
- Как зарегистрировать интерцептор в конфигурации?
Через реализацию `WebMvcConfigurer`:

```

1 @Configuration
2 public class WebConfig implements WebMvcConfigurer {
3     @Override
4     public void addInterceptors(InterceptorRegistry registry) {
5         registry.addInterceptor(new MyInterceptor());
6     }
7 }

```

- Какие методы есть у `HandlerInterceptor`?
 - `preHandle()` — вызывается до контроллера.
 - `postHandle()` — вызывается после контроллера, до View.
 - `afterCompletion()` — вызывается после рендеринга View.

4.11 @Controller vs. @RestController

- Чем отличается @Controller от @RestController?

@Controller возвращает имя View (например, JSP), а @RestController — это сокращение для @Controller + @ResponseBody, автоматически возвращает JSON.

- Как вернуть JSON-ответ с помощью @ResponseBody?

Пример использования в контроллере:

```
1 @Controller
2 public class UserController {
3     @GetMapping("/user/{id}")
4     @ResponseBody
5     public User getUser(@PathVariable Long id) {
6         return userService.findById(id);
7     }
8 }
```

- Какие HTTP-статусы следует использовать в REST (ResponseEntity)?

Пример с управлением статусом:

```
1 @GetMapping("/user/{id}")
2 public ResponseEntity<User> getUser(@PathVariable Long id) {
3     User user = userService.findById(id);
4     if (user == null) {
5         return ResponseEntity.status(HttpStatus.NOT_FOUND).build();
6     }
7     return ResponseEntity.ok(user);
8 }
```

Часто используемые статусы: 200 OK, 201 Created, 400 Bad Request, 404 Not Found, 500 Internal Server Error.

4.12 Транзакции в Spring: как работают, свойства

- Какие параметры можно задать (isolation, propagation)?

Аннотация @Transactional поддерживает:

- `isolation` — уровень изоляции (например, `READ_COMMITTED`, `SERIALIZABLE`).
 - `propagation` — поведение вложенных транзакций (`REQUIRED`, `REQUIRES_NEW` и др.).
 - `timeout`, `readOnly`, `rollbackFor`.
- В каких случаях транзакция откатывается (`rollback`)?
По умолчанию — при непроверяемых исключениях (`RuntimeException`).
Для проверяемых — нужно указывать `rollbackFor`.

4.13 Жизненный цикл сущности в Hibernate

Состояние	Описание
Transient	Объект создан с помощью <code>new</code> , не привязан к Hibernate , не сохранён в БД.
Persistent	Объект связан с активной <code>Session</code> Hibernate. Все изменения отслеживаются.
Detached	Объект раньше был persistent , но <code>Session</code> закрыта. Не отслеживается.
Removed (optional)	Сущность помечена на удаление. Физически будет удалена при <code>flush</code> / <code>commit</code> .

Таблица 1: Состояния объектов в Hibernate

- Какие аннотации используются для маппинга сущностей в Hibernate?

```

1 @Entity
2 @Table(name = "users")
3 public class User {
4     @Id
5     @GeneratedValue(strategy = GenerationType.IDENTITY)
6     private Long id;
7

```

```
8  @Column(name = "username")
9  private String username;
10 }
```

- Как Hibernate отслеживает изменения в сущностях (Dirty Checking)?
Hibernate сохраняет состояние сущности при загрузке, и при коммите сравнивает с текущим состоянием, обновляя базу только при изменениях.

4.14 Связи между сущностями в Hibernate

- Какие типы связей между сущностями поддерживает Hibernate?
`@OneToOne`, `@OneToMany`, `@ManyToOne`, `@ManyToMany`.
- Как работают `@OneToMany`, `@ManyToOne`, `@ManyToMany`?
 - `@ManyToOne` — много сущностей ссылаются на одну (внешний ключ).
 - `@OneToMany` — коллекция сущностей, связанных с одной.
 - `@ManyToMany` — связь через промежуточную таблицу.
- В чем разница между `EAGER` и `LAZY` загрузкой?
`EAGER` загружает сразу связанные данные, `LAZY` — отложенная загрузка при обращении.

4.15 Что такое Hibernate

- Как Hibernate связан с JPA?
Hibernate — популярная реализация спецификации JPA.
- Какие есть стратегии загрузки данных (`EAGER`, `LAZY`)?
Определяют момент загрузки связанных данных: сразу или по требованию.

- Как работает кэширование в Hibernate?

Используются:

- Первый уровень кэш — сессия (кэш на время транзакции).
- Второй уровень кэш — глобальный, опционально (например, Ehcache).

4.16 Что такое JPA

- Какие есть основные аннотации?

`@Entity`, `@Table`, `@Id`, `@Column`, `@GeneratedValue` — используются для маппинга сущностей на таблицы базы данных.

```
1 @Entity
2 @Table(name = "users")
3 public class User {
4     @Id
5     @GeneratedValue
6     private Long id;
7
8     @Column(name = "username")
9     private String name;
10 }
```

- Как работают каскадные операции (`CascadeType`)?

Они определяют, какие действия (`persist`, `merge`, `remove` и др.) должны быть применены автоматически к связанным сущностям.

- Что такое `EntityManager` и `@PersistenceContext`?

`EntityManager` — основной API для работы с сущностями в JPA.

`@PersistenceContext` внедряет `EntityManager` в компонент:

```
1 @PersistenceContext
2 private EntityManager em;
```

4.17 Проблема LazyInitializationException

- Почему возникает эта ошибка?

Ошибка `LazyInitializationException` возникает, когда происходит обращение к LAZY-загруженному полю вне контекста открытой сессии/транзакции.

- Как избежать проблем с `LazyInitializationException`?

- Использовать `fetch = FetchType.EAGER` (нежелательно для больших графов).
- Открывать сессию до конца обработки запроса (Open Session in View).
- Явно загружать данные с помощью `JOIN FETCH`.

- Как правильно работать с LAZY-загрузкой внутри транзакции?

Загрузка связанных сущностей должна происходить внутри метода с аннотацией `@Transactional`.

4.18 Проблема N+1 в Hibernate

- Пример возникновения проблемы:

Если при загрузке списка пользователей каждый раз отдельно загружается их профиль:

```
1 List<User> users = em.createQuery("from User", User.class).getResultList();
2
3 for (User u : users) {
4     System.out.println(u.getProfile().getBio());
5 }
```

- Как избежать N+1 проблемы?

Использовать `JOIN FETCH`:

```
1 List<User> users = em.createQuery(
```

```
2 "select u from User u join fetch u.profile", User.class).getResultList  
   ();
```

- Какие способы решения существуют?
 - Использовать JOIN FETCH.
 - Настроить batch-fetching в Hibernate.
 - Использовать DTO-проекции.

4.19 Как работает Spring Security

- Как настроить аутентификацию через базу данных?
Создать таблицы `users` и `authorities`, настроить `UserDetailsService` или использовать JDBC-конфигурацию:

```
1 @Autowired  
2 DataSource dataSource;  
3  
4 @Bean  
5 public SecurityFilterChain filterChain(HttpSecurity http) throws  
   Exception {  
6     http.jdbcAuthentication().dataSource(dataSource);  
7     return http.build();  
8 }
```

- Как реализовать JWT-аутентификацию в Spring Security?
Добавить фильтр, который проверяет и валидирует JWT из заголовка `Authorization`, и добавляет пользователя в `SecurityContext`.

4.20 CSRF и XSS в Spring Security

- Как Spring Security защищает от CSRF и XSS?
CSRF — генерирует токен и проверяет его в каждом запросе.

XSS — фильтрует ввод и можно дополнительно настроить Content Security Policy (CSP).

- Почему важно не отключать CSRF-защиту в REST API?

Даже если REST не использует формы, браузерные атаки (например, через XHR) возможны при авторизации сессией. Лучше использовать stateless и JWT.

- Как настроить Content Security Policy (CSP)?

Добавить заголовок CSP:

```
1 http.headers().contentSecurityPolicy("script-src 'self'");
```

4.21 Spring Actuator

- Какие основные endpoints предоставляет Actuator и для чего они нужны?
 - `/actuator/health` — проверка статуса приложения (живое/нет).
 - `/actuator/metrics` — системные метрики: память, CPU, загрузка.
 - `/actuator/info` — произвольная информация о приложении (например, версия).
- Как защитить endpoints Actuator в продакшене?
 - Отключать ненужные endpoints в `application.yml`.
 - Ограничить доступ с помощью Spring Security.
 - Разрешить доступ только из внутренней сети.

4.22 Четыре золотых сигнала мониторинга

- **Latency** — время отклика на запрос. Рост латентности — признак перегрузки.

- **Traffic** — объем запросов или нагрузка (например, RPS). Увеличение трафика требует масштабирования.
- **Errors** — процент ошибочных запросов (4xx/5xx). Указывает на сбои.
- **Saturation** — насколько система приближается к лимитам (CPU, память, очереди).

4.23 SLI, SLO, SLA

- **SLI (Service Level Indicator)** — числовой показатель качества сервиса (например, процент успешных запросов).
- **SLO (Service Level Objective)** — целевое значение SLI (например, 99.9% успешных запросов за месяц).
- **SLA (Service Level Agreement)** — формальное соглашение с заказчиком, основанное на SLO.
- Как квантили помогают в определении SLI?
Квантили (например, p95, p99) показывают, сколько процентов запросов укладываются в заданную задержку, и могут быть SLI по латентности.
- Пример SLO:
99.9% всех запросов к `/api/*` должны завершаться менее чем за 300 мс за последние 30 дней.

4.24 Histogram и Summary в Prometheus

- **Histogram** — агрегирует данные по заранее определённым bucket'ам и позволяет вычислять квантили на стороне Prometheus.
- **Summary** — квантили вычисляются на стороне клиента и не могут быть агрегированы между экземплярами.

- **Вывод:** Histogram подходит для распределённых систем, где важна агрегация.

4.25 Prometheus + Grafana: архитектура и push/pull-модель

- Prometheus — система мониторинга и хранения метрик (в pull-модели).
- Grafana — система визуализации, строит графики по данным Prometheus.
- **Pull-модель:** Prometheus опрашивает приложения по HTTP (встроенный HTTP-сервер с метриками).
- **Push-модель:** приложения отправляют метрики в промежуточный push-gateway, который опрашивается Prometheus.
- **Разница:**
 - Pull проще и предпочтительнее для долгоживущих сервисов.
 - Push — полезен для короткоживущих задач (batch jobs), где нет смысла держать HTTP-сервер.

5. Sync/async.

5.1 Что такое OpenAPI

- Какие проблемы он решает?
OpenAPI (ранее Swagger) позволяет стандартизировать описание REST API, обеспечивая единый формат для клиентов, документации и генерации кода.

- Как OpenAPI помогает в документировании REST API?
OpenAPI-спецификация описывает эндпоинты, методы, параметры, модели данных и коды ответов, что позволяет:
 - Автоматически генерировать документацию.
 - Упростить интеграцию для клиентов.
 - Проверять корректность запросов и ответов.
- Какие инструменты используются для генерации документации в Spring?
springdoc-openapi, Swagger UI, OpenAPI Generator.

5.2 Модель OSI

Модель OSI (Open Systems Interconnection) — это семиуровневая концептуальная модель, которая описывает, как данные передаются по сети от одного устройства к другому. Она помогает структурировать и стандартизировать работу сетевых протоколов.

- Уровни и примеры протоколов:
 1. **Физический** — Ethernet (физические соединения).
 2. **Канальный** — Ethernet, PPP.
 3. **Сетевой** — IP, ICMP.
 4. **Транспортный** — TCP, UDP.
 5. **Сеансовый** — NetBIOS, RPC.
 6. **Представления** — TLS, JPEG, ASCII.
 7. **Прикладной** — HTTP, FTP, DNS, SMTP.

5.3 Что такое HTTP: работа, параметры

- Какие уровни модели OSI охватывает HTTP?

HTTP — прикладной уровень (7), использует TCP с IP (уровни 4 и 3).

- Как структурирован HTTP-запрос и HTTP-ответ?

Запрос:

- Стартовая строка (метод, путь, версия).
- Заголовки (headers).
- Тело (необязательно).

Ответ:

- Статусная строка (версия, код, сообщение).
- Заголовки.
- Тело (обычно JSON, HTML и т.д.).

5.4 Что такое REST API

REST API (Representational State Transfer API) — это архитектурный стиль для построения веб-сервисов, в котором используется HTTP и принципы REST для взаимодействия между клиентом и сервером.

- Почему REST API предпочтительнее использования одного метода (POST)?

REST использует стандартные HTTP-методы для CRUD-операций (GET, POST, PUT, DELETE), что делает API:

- Предсказуемым и читаемым.
- Кэшируемым (например, GET).
- Совместимым с HTTP-инфраструктурой.

- Как REST API связан с HTTP-методами и ресурсами?

Каждый ресурс (например, `/users`) управляется через соответствующие HTTP-методы:

- `GET /users` — получить список.
- `POST /users` — создать.
- `PUT /users/1` — обновить.
- `PATCH /users/1` — частично обновить.
- `DELETE /users/1` — удалить.

5.5 Коды состояния HTTP

- Примеры кодов:

- **2xx (успех)**: 200 OK, 201 Created.
- **3xx (перенаправление)**: 301 Moved Permanently, 302 Found.
- **4xx (ошибка клиента)**: 400 Bad Request, 404 Not Found.
- **5xx (ошибка сервера)**: 500 Internal Server Error, 503 Service Unavailable.

- Как обрабатывать ошибки в REST API?

- Возвращать корректные коды (например, 404, если ресурс не найден).
- Использовать `ResponseEntity` с телом ошибки:

```
1 return ResponseEntity.status(404)
2                          .body("User not found");
```

- Логировать ошибки и не раскрывать чувствительную информацию.

5.6 Синхронное и асинхронное программирование

- Какие есть преимущества и недостатки асинхронности?

Преимущества:

- Более высокая производительность при работе с I/O.
- Не блокирует поток на ожидание.

Недостатки:

- Сложность отладки и сопровождения.
 - Неочевидное управление потоками и ошибками.
- Как асинхронность реализована в Java?
 - `Future`, `CompletableFuture`, `ExecutorService`.
 - В Spring — аннотация `@Async` и `TaskExecutor`.

5.7 Что такое Circuit Breaker

Circuit Breaker (предохранитель, «автомат защиты») — это шаблон отказоустойчивости, который используется для предотвращения каскадных сбоев в распределённых системах.

- Как Circuit Breaker предотвращает каскадные сбои?

Если внешняя система постоянно возвращает ошибки, Circuit Breaker временно "разрывает" соединение и не посылает новые запросы, давая системе восстановиться.

- Какие есть реализации?

`Resilience4j`, `Hystrix` (устарел).

- Как настроить Circuit Breaker в Spring?

С использованием Spring Boot + Resilience4j:

```
1 @CircuitBreaker(name = "myService", fallbackMethod = "fallback")
2 public String callExternal() {
3     return restTemplate.getForObject(...);
4 }
```

5.8 Что такое Rate Limiter и зачем он нужен

- Rate Limiter ограничивает количество запросов к ресурсу за определённое время. Помогает:

- Защититься от перегрузки.
- Гарантировать честный доступ к ресурсам.

- Где применяется Rate Limiter?

И на клиенте, и на сервере:

- **Клиент:** чтобы не перегружать внешний API.
- **Сервер:** чтобы защититься от DDoS или чрезмерной активности.

5.9 Гарантии доставки сообщений

- **At most once** — сообщение может не дойти, но не повторится. Быстро, но ненадёжно.
- **At least once** — сообщение точно дойдёт, но может повторяться. Нужна идемпотентность.
- **Exactly once** — сообщение дойдёт ровно один раз. Самая сложная гарантия.
- Применение:

- **At most once** — UDP, метрики, телеметрия.
- **At least once** — Kafka по умолчанию.
- **Exactly once** — денежные переводы, бухгалтерия.

5.10 Идемпотентные операции

Идемпотентные операции — это такие операции, результат которых не меняется при повторном выполнении с теми же входными данными.

- Примеры:

- **Идемпотентные:** GET /user/1, DELETE /user/1, PUT /user/1.
- **Неидемпотентные:** POST /user (создание каждый раз нового).

- Как помогает достичь **exactly once**?

Если операция идемпотентна, то её повторное выполнение не изменяет состояние — это позволяет безопасно обрабатывать повторные доставки в системах с гарантией **at least once**.

5.11 Что такое Kafka

Apache Kafka — это распределённая платформа потоковой передачи сообщений (streaming platform), предназначенная для передачи, хранения и обработки больших объёмов данных в реальном времени.

- Как Kafka обеспечивает обмен сообщениями?

Kafka — это распределённая очередь сообщений. Producer пишет в топики (topics), каждый из которых делится на партиции. Consumer читает данные из партиций.

- Как Kafka гарантирует доставку сообщений?

Через:

- Репликацию партиций между брокерами.
- Настройку `acks` у `producer`.
- Управление смещениями (`offset`) у `consumer`.

5.12 Как работает `consumer group` в `Kafka`

- Что происходит, если количество потребителей не совпадает с количеством партиций?
 - **Партиций больше:** некоторые потребители читают несколько партиций.
 - **Потребителей больше:** лишние потребители простаивают.
- Почему два `consumer` из одной группы не могут читать одну партицию? Каждая партиция может быть назначена только одному `consumer` в группе для обеспечения порядка чтения.

5.13 Что такое `commit offset` в `Kafka`

В `Apache Kafka` `offset` — это уникальный идентификатор позиции сообщения в партиции.

А `commit offset` — это процесс сохранения текущей позиции, до которой `consumer` успешно обработал сообщения.

- Как работает `auto-commit` и какие у него могут быть проблемы?
`auto-commit=true` по умолчанию — `offset` фиксируется автоматически. Может привести к потере данных при сбое между `commit` и обработкой.
- Почему `Kafka` не удаляет сообщения после `commit`?
`Kafka` хранит сообщения по времени (`retention`) или размеру, а не по `offset`.

Это позволяет нескольким группам читать одни и те же сообщения независимо.

5.14 Настройки `acks` у `producer` и гарантии доставки

- Чем отличается `acks=0`, `acks=1`, `acks=all`?
 - `acks=0` — не ждёт подтверждений, возможна потеря.
 - `acks=1` — ждёт от лидера, возможна потеря при сбое до репликации.
 - `acks=all` — ждёт подтверждения от всех in-sync реплик.
- Как параметр `min.insync.replicas` связан с надёжностью?

Устанавливает минимальное число реплик, которые должны подтвердить запись при `acks=all`. Если их меньше — запись будет отклонена.

5.15 Отказоустойчивость и масштабируемость `Kafka`

- Как работает репликация партиций?

Каждая партиция имеет лидера и несколько реплик. Только лидер обрабатывает запись/чтение, остальные — резервные копии.
- Что такое `retention policy` и как она влияет на хранение?

`Kafka` хранит сообщения независимо от чтения. Политика `retention`:

 - `retention.ms` — по времени.
 - `retention.bytes` — по размеру.

Позволяет настраивать долговременное хранение или очистку сообщений.

5.16 Что такое loose coupling в контексте асинхронного взаимодействия

В контексте асинхронного взаимодействия loose coupling (слабая связность) означает, что компоненты системы (например, микросервисы) взаимодействуют друг с другом без жёсткой зависимости. Они не знают напрямую друг о друге, а обмениваются сообщениями через посредника — брокера сообщений (например, Kafka, RabbitMQ).

- Как брокеры сообщений помогают уменьшить связанность между сервисами?

Брокеры выступают посредниками: сервисы не знают напрямую друг о друге, а обмениваются сообщениями через брокер. Это снижает жёсткую связанность.

- Пример жесткой и слабой связанности в коде:

Жесткая связанность:

```
1 OrderService orderService = new OrderService();
2 orderService.createOrder();
3 PaymentService paymentService = new PaymentService();
4 paymentService.processPayment();
```

Прямые вызовы — высокая связанность.

Слабая связанность через брокер:

```
1 messageBroker.publish("order.created", orderData);
2 messageBroker.subscribe("order.created", event -> processPayment(event))
   ;
```

Сервисы взаимодействуют через сообщения.

5.17 Проблемы консистентности при работе с несколькими источниками данных

- Пример кода, где сообщение в Kafka отправляется до/после коммита транзакции и риски:

```
1 producer.send(kafkaMessage);  
2 dbTransaction.commit();  
3  
4 dbTransaction.commit();  
5 producer.send(kafkaMessage);
```

- Почему нельзя гарантировать строгую консистентность без дополнительных механизмов?

Разные системы управляют транзакциями независимо. Без координации возможны рассинхронизации.

5.18 Как работает двухфазный коммит (2PC)

- Гарантии доставки: **at least once**, но не **exactly once**
2PC гарантирует согласованность двух и более систем, но может приводить к повторной отправке сообщений.
- Преимущества и недостатки:
 - + Гарантирует согласованность.
 - - Высокая задержка и блокировка ресурсов.
 - - Сложность реализации и возможные дедлоки.

5.19 Как работает Transactional Outbox

- Гарантии доставки: обычно **at least once**
Сообщения сохраняются в отдельной таблице в той же транзакции, что

и изменения данных, а потом асинхронно отправляются.

- Преимущества и недостатки:

- + Избегает проблем 2PC.
- + Высокая отказоустойчивость.
- - Возможны дубликаты, требующие идемпотентной обработки.

5.20 Что такое JSON, преимущества и недостатки

- Почему JSON популярен в REST API?

Простота, человекочитаемость, поддержка в большинстве языков, гибкая структура.

- Проблемы при работе с типами данных:

Например, `long` может быть больше, чем максимальное значение `integer` в JavaScript, что ведёт к потере точности.

5.21 Что такое Protocol Buffers и как они работают

- Как определяется схема данных в `.proto`-файле?

Через описание сообщений, полей с типами и номерами:

```
1 message Person {  
2     int32 id = 1;  
3     string name = 2;  
4     repeated string emails = 3;  
5 }
```

- Какие типы полей поддерживаются?

`optional`, `required` (в старых версиях), `repeated`, `map`.

5.22 Что такое Avro и как он работает

- Как Avro использует схемы?

Схема JSON описывает структуру данных, используется при записи и чтении для валидации и сериализации.

- Разница между схемой для записи и чтения:

Позволяет эволюцию схемы, например, добавлять новые поля (чтение старой записи новой схемой).

5.23 Что такое прямая и обратная совместимость схем

- Примеры нарушений обратной совместимости:

Удаление поля, изменение типа поля.

- Как Protocol Buffers и Avro помогают поддерживать совместимость?

Используют номера полей и правила изменения схем, чтобы новые версии могли читать старые данные и наоборот.

5.24 Как обеспечить совместимость при добавлении новых полей в JSON-схему

- Аннотации Jackson, которые помогают:

`@JsonIgnoreProperties(ignoreUnknown = true)` — игнорирует неизвестные поля.

`@JsonProperty` — управляет сериализацией и десериализацией.

- Почему важно помечать новые поля как `optional` в бинарных форматах?

Чтобы старые клиенты могли работать с новыми сообщениями без ошибок.